

Quality Assurance of Jobs.ps

1st Dr. Samer AL-Zain
dept. Software engineering
Birzeit University
Ramallah, Palestine
szain@birzeit.edu

2nd Imadeddin Salama
dept. Software Engineering
Birzeit University
Ramallah, Palestine
ce.emad1999@gmail.com

3rd Duha Jarrar
dept. Software Engineering
Birzeit University
Ramallah, Palestine
duha.jarrar@gmail.com

4th Mohammed Abuqare
dept. Software engineering
Birzeit University
Ramallah, Palestine
abuqare@gmail.com

Abstract—Software quality assurance and testing play a vital role in increasing the quality of software and ensuring that software meets the requirements specification. In Palestine, there are several real-world software applications that are being used by hundreds of thousands of users. The goal of this research is to assess the quality of Jobs.ps, one of those local websites, by designing and implementing a comprehensive testing strategy, then executing it, and analyzing the results in which it reflects the quality assurance of the local software industry. The results of executing the test plan show that there are 24 bugs produced from 42 manual and automated test cases, and the selected website has failed to comply with the designed performance and load tests. This provides an indication that the selected website and similar local websites do not follow a systematic quality assurance strategy to ensure the quality of their proposed functionalities.

Index Terms—Software quality assurance, Jobs.ps, test plan, performance testing, load testing, test automation and manual testing

I. INTRODUCTION

The main objectives of software quality assurance are to increase the quality of software and software development life cycle including increasing the software quality itself by reducing the number of defects and bugs, development process and code quality. Quality assurance helps finding errors before they occur, which helps the developers fix them as early as possible and avoid future damages. Applying systematic software quality assurance would help meeting the needs and specifications of the software being developed in the desired quality. In software development fixing a bug may lead to open more bugs at the other end. Therefore, a systematic process of quality assurance and testing would ensure the quality of the software during evolution and maintenance. Defining a clear testing processes and procedures in multiple stages including regression testing, automaton testing and manual end-to-end testing to assure the correctness, reliability and the quality of the software.

The goal of this research is to design and implement a comprehensive test plan and strategy to evaluate the quality of an already implemented software. In the next section, we will provide a step by step strategy the selecting real-world software, gathering the requirement, selecting software management tool, selecting test management tool, automation testing tool, and performance testing tool by taking into consideration the selected software, the managing team, the software domain, and technology. In the third section, we will

design a test plan. In section 4, we will develop the test cases of the main functionalities. In section 5, we will discuss how we managed the test cases, test plan and the execution results. In section 6, we will analyze the results of the execution. In section 7, we will discuss the test automation project setup and how it integrates to test management tool and execution results. In section 8, we will discuss the setup of performance testing tool, different performance types, performance metrics and the execution results and analysis. In section 9, we will provide some quality enhancement suggestions. And the last section is the conclusion.

II. METHODOLOGY

In this section, we will discuss the strategy and methodology of selecting the software to be tested and the quality assurance tool-set for managing the end-to-end quality assurance and testing of that software based on local industry and recommendations.

A. Real world software

We have selected one of the well-known websites in Palestine and the aim is to test the quality of local software in Palestine.

Jobs.ps is the first and leading job posting and employment websites in Palestine, it is a well-known web application, where most local companies post their vacancies. It is being used by companies and job seekers where companies can post their open vacancies and job seekers can search for job opportunities and subscribe to job alerts in a simple form, Fig. 1 shows a screenshot of the website main page.

Jobs.ps has over 730,000 visits according to latest last three months statistics of SimilarWeb, and over 80% of traffic are from Palestine, 50% of which are direct traffic and about 40% are from organic search and the remaining 10% are mostly from social media [1]. 55% the physical devices are from mobile and the other 45% are from desktop [1].

Jobs.ps Ltd, located in Ramallah. As a company, it has a small team of non-tech 2-5 employees and has SLA with one of the local development companies to manage and maintain the website. According to Jobs.ps Ltd, it has around one million visits per month, more than 100,000 registered job-seekers, and around 40,000 posted jobs, and almost 3,000 active employer accounts [2].

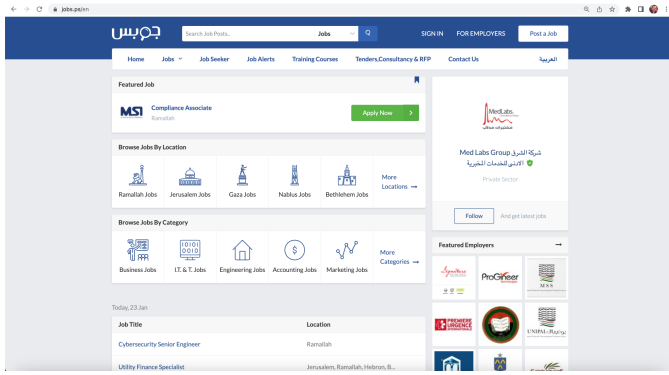


Fig. 1: A screenshot of jobs.ps home page

Jobs.ps was chosen for the study due to several reasons. Firstly, to understand and provide insights about local industry software quality; Secondly, it is very well-known and actively used. Thirdly, the company has shown high interest to check the quality of their website. Finally, as per our initial assessment there were several areas of the website that we can test and improve.

B. Requirements

The requirements were elicited based on Term of Reference annex that was provided to the development companies to implement the current version of website. We wrote the requirements and divided them into functional and nonfunctional requirements in which we will implement the test cases and testing accordingly. All functional and nonfunctional requirements are listed in Appendix A.

C. Software Management Tool

Scrum is a widely used framework as part of agile project management, in highlight of State of Agile Annual Report of 2022, Scrum is most dominant agile framework, 87% of respondents use Scrum at the team level - up from 58% over the last tree years [3], see Fig 2.

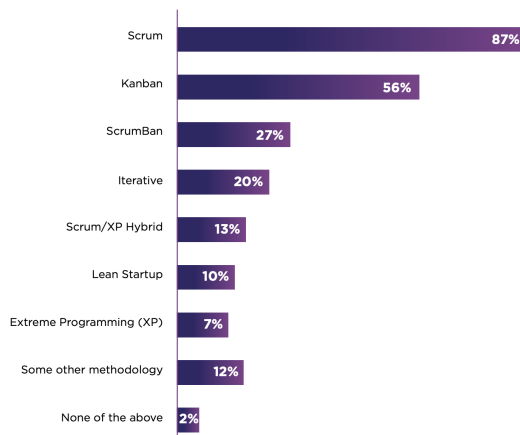


Fig. 2: Top Use of Agile Frameworks, 2022 [3]

In accordance with lately conducted industry studies in Palestine IT sector, almost 50% of companies use agile as

their primary software development life cycle [4]. Therefore, in order to make work smooth and accelerate the onboarding process in case Jobs.ps decided to work with another provider or build an internal tech team then they would have a good talent pool who are familiar with this tool. JIRA/scrum software management tool was used to document the requirements as depicted in Fig. 3. JIRA is one of the most used tools in Palestine market according to [3], in which most companies in Palestine use agile methodology and scrum management framework in particular. Most of them have adapted JIRA in their software management projects and product management tools.

Sprints	15	16	17	18	19	20	21	22	23
JT-2 Account									
JT-4 Dashboard									
JT-5 The user shall be able to create new job alerts									
JT-6 The user shall be able to search among opening jobs, through the search phrase, location, category and job type.									
JT-7 The user shall be able to view his saved jobs in the system.									
JT-8 The user shall be able to view his job application on each job he sent an application to.									
JT-9 The user shall be able to update his account basic information (name, phone number, CV attached, password, education, skills, work experience, projects)									
JT-10 The user shall be able to upload and update his CV.									
JT-11 Saved Jobs									
JT-12 Resume Builder									
JT-13 Browse Jobs									
JT-14 Following									
JT-15 Job									
JT-16 Search									
JT-17 Employer									
JT-18 Home									
JT-19 My Job Watcher									
JT-20 Ask for Recommendation									
JT-21 Employer's Requests									

Fig. 3: JIRA for managing the requirements

D. Test Management Tool

Given that we have selected JIRA as software management tool, and in order to keep everything in one place, Zephyr Scale Cloud - Test Management for Jira has been selected, it supports traceability matrix between requirements and test cases feature, it integrates with Cucumber BDD for aligning teams using Behaviour-Driven Development (BDD)/Gherkin language and it also supports REST API integration for uploading cucumber reports and some other commonly used frameworks [5]. The BDD makes it easier for non-tech people to understand the testing cases as it is written in natural language. Given that we have a non-tech team managing jobs.ps website that would be easier for them to understand the automated tests. The BDD/Gherkin steps can be implemented using selenium testing scripts and can be automated with cucumber/junit4 and then upload the generated cucumber reports after the execution to the test management tool using REST APIs simple shell script.

1) *Zephyr Scale - Test Management:* Zephyr Scale Cloud was known as Test Management for Jira Cloud (TM4J Cloud) that integrates with Jira and helps trace test cases in a central location (Jira). According to Zephyr Scale documentation [5], it is the number one customer rated QA and test management app for JIRA.

a) *Test Cases:* According to [5], a test case is “a test artifact developed for a particular testing need, such as to exercise a particular program path or to verify compliance with a specific requirement”. It supports creating and importing

test cases, defining parameters to be used among test cases, reusing Test Cases with call to test when defining steps, for example if login test case is required as precondition to another test case in that particular step, you can link login test case using call to test action. It also supports version control, where test cases can be versioned during the evolution of the software, for example if you updated the name of the actions on the website you can create new version for those changes where you only change the name of the actions and keep the old version as is. So, if you need to revert back, you still have the test cases with the old names of the previous version.

Each test case can be written in three types: Plain text, Step-by-Step where you can describe each step separately by defining test data and expected result or use Gherkin Behavior-Driven Development (BDD) where you write the test case using Gherkin language. The BDD can be exported later to be used with automation testing. Test cases can be linked to Jira issues and vice versa where you can view tractability between issues and test cases. You can create it from the issue or test case. It also has flexibility on running test cases, so the team can execute a single test case directly from Jira issue without running the whole test cycle.

b) *Test Cycles*: Test cycle is a group of a set of test cases that are designed to achieve a specific goal, for example regression testing, smoke testing or end-to-end testing. A group of test cycles can form a test plan. The test cycle helps tracking testing progress as depicted in Fig. 5, and compare between estimated effort and actual testing time. Running test

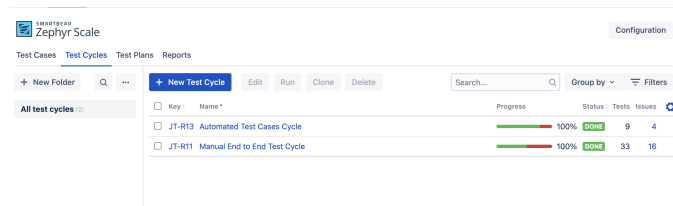


Fig. 4: Zephyr Test Cycles

cases will open Test Player, the test cases will be executed step by step after clicking on run button. Each step can be linked to issues or attached files or descriptions of the test results and issues as depicted in Fig. 3, where in Step 4 there was an error, the issue was created, and a screenshot was attached to the execution steps. That will help the developers to understand what was the error and in which step it happened.

c) *Test Plan*: Test Plan according to [5] is a set of test cycles planned to achieve a higher goal, for example release testings. Each test plan can have multiple test cycles where each test cycle has a set of test cases and each test case has set of execution results and issues related. The Fig. 6 provides an overview of test plan.

d) *Test Automation and Continues Integration*: Zephyr supports top test automation frameworks including Cucumber, Junit, TestNG, PyTest and Junit4 and other custom formats using REST API endpoints, where it can be used also with

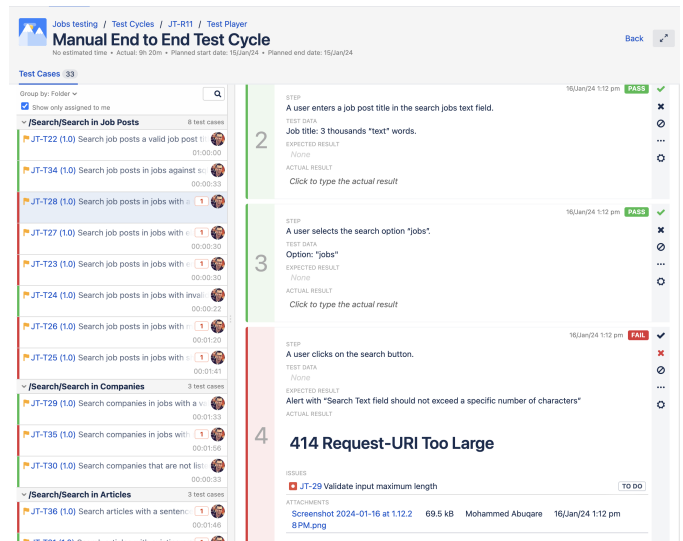


Fig. 5: Zephyr Test Player

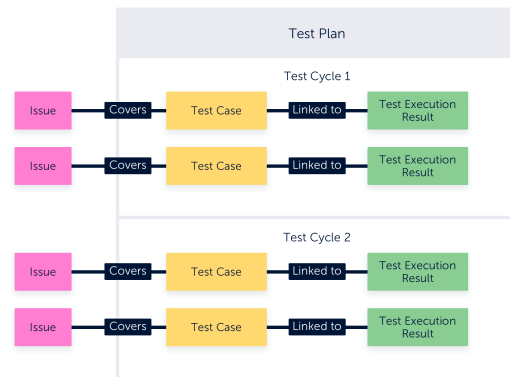


Fig. 6: Zephyr: Overview of test plan association with test cases and issues [5]

Continue Integration frameworks like Jenkins and Bamboo. Cucumber was used in our case and uploaded using REST API, it can be integrated with any CI frameworks, Fig 7 show an example of integration using Jenkins and Cucumber.

e) *Reports and Analysis*: Zephyr scale cloud has 31 competences reports where 27 for executions reports and 4 for traceability/coverage. Test-Execution Reports provide overview of testing effort, including quality and test execution progress for example Test execution effort by tester. Traceability Reports enable testers to review test coverage details for example traceability report which is a table of tractability between issues, test cases and test executions. We will use some reports in the executions section.

2) *Gherkin Behavior-Driven Development (BDD)*: Gherkin is a language that is being used to describe test cases using simple keywords in which it makes a common understanding between tech and non-tech people. BDD file ends with .feature extension, which is a plain text file. Each line of it has to start

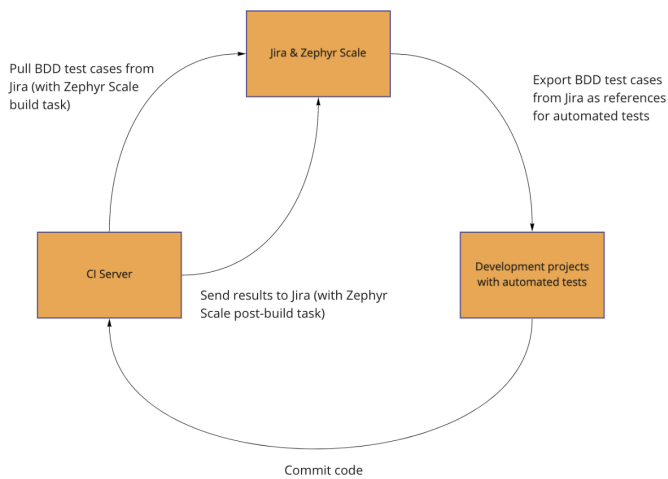


Fig. 7: Cucumber & Jenkins Integration [5]

with a keyword from Gherkin language. It is designed to form a flow or steps in the following sequence: 'Given', 'When' and 'Then' connected with 'And'. Other keywords are used to provide more information to the test steps, but in Zephyr there is no need to use other keywords, as these keywords will be generated automatically from Zephyr using export BDD. Keyword 'Examples' is used to provide more data examples to reuse same steps of BDD. The examples will be automatically generated using Cucumber and Junit. We will provide more examples in the automation section.

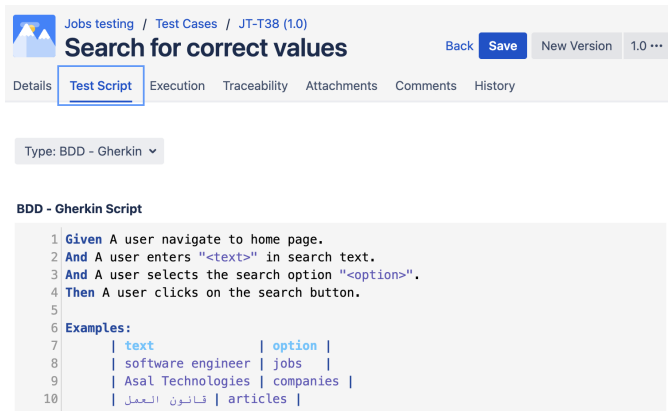


Fig. 8: An example of BDD in Zephyr

Fig. 8 shows an example of using BDD in Zephyr where “<text>” and “<option >” are dynamically provided by “Examples” keyword during runtime.

E. Performance Testing Tool

Performance testing tools are used to perform different kind of performance testing including Load, Stress, Smoke, Soak and Spike testing depending on nonfunctional requirement to be tested. For example, Stress test is used to test the performance of the software under continuously increasing the number of active users till software reaches the limit of what

it can handle. Load testing is used to test the performance of the software under continuous and consistent load for a period of time that the software is expected to handle. Soak testing is used to provide persistent load to simulate production load similar to load testing to some extends. Spike testing is used to test seasonal load that occurs during a hype in sales or instant increase of load.

1) *Jmeter*: Jmeter is an open source Apache application was designed for testing web applications then expanded to support other fields. It is important to note that Jmeter is not a browser. Thus, it does not support cookies and executes JavaScript codes. It was built to perform test at protocol level. The following are some advantages of using Jmeter [6].

- Its ability to perform load and performance testing using different set of protocols, applications and servers types.
- Ability to use command lines and GUI at the same time.
- A complete generated report ready to use HTML report.
- Multi-threading framework which allows concurrent sampling by many threads and performing different functions using thread groups.
- Supports a wide range of plugins and extensions.
- It also supports recording Test using browser over proxy.

Jmeter was selected to perform the performance testing of jobs.ps, given that it is open-source, and simple to install. In addition, it can perform the test needed, there were some attempts to use LoadNinja and we spoke with SmartBear Sales team and met with them online in order to convince them to provide us the one month trial plan, but unfortunately they did not.

2) *Setup*: HTTP(S) Test Script Recorder was used to record browser requests by setting up proxy server and pointing the browser to use that proxy server. Given that it was using https protocol an SSL certificate was needed to be registered in the browser, it is located under /bin directory, it will be automatically generated once running the recorder proxy server [6]. After configuring the browser, 5 pages of jobs.ps were recorded to simulate the usage of real users to perform performance testing according to test plan. A thread group was used to determine the Number of threads (users), Ramp-up period (seconds) and Duration (seconds) or Loop count. The configuration depends on the test that needs to be performed. We revisit again in the execution section of performance testing. Fig. 9 shows an example of Jmeter configurations and setup. Later on, a command line was used to generate the performance report.

3) *Dashboard Report*: Jmeter dashboard report provides multiple performance statistics in the form of tables and graphs, and APDEX (Application Performance Index). A summary of successful and failed requests and many others, we will present some of the performance test results later according to the performance test needed in the test plan.

Fig. 10 shows a sample of dashboard report.

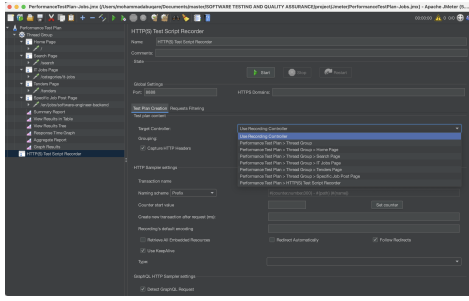


Fig. 9: Jmeter setup and configuration.

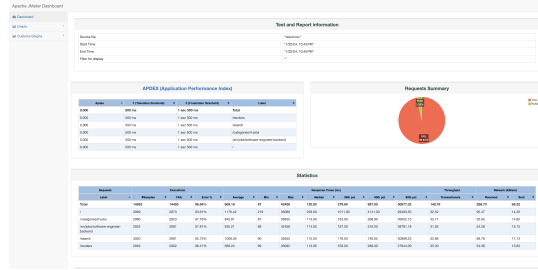


Fig. 10: A sample of Jmeter Dashboard Report.

III. TEST PLAN

A. Introduction

The objective of this test plan is to organize the testing process of Jobs.ps in terms of functionalities and quality attributes, it shows the testing strategy and methodology that will be used, the testing process time management, resources, the exit criteria and approvals, and other needed information.

B. Scope

1) *Inclusions*: This test plan covers functional, performance, and load testing.

2) *Exclusions*: The test plan will not include usability testing or other functionalities than the aforementioned in the Inclusions section due to time limitations.

C. Test Objectives

The test objectives of this test plan are to test the functional requirements and ensure that the system is reliable and efficient to do all what it was created for. Also to test the performance and security of the system to ensure the system can handle concurrent requests and handle threats that may face from users. In addition, we will test the usability of the system and how its easy to use through memorability and effectiveness.

1) *Functional Testing Objectives*: The functional testing will include testing different scenarios such as

- Verify login for both job-seeker and employer.
- Validate the entered personal information while creating a resume using the resume builder feature.
- Validate job matcher.
- Validate browsing jobs by location.
- Verify follow job provider feature.

- Verify save jobs.
- Validate apply for a job.
- Validate search in job posts, companies, and articles.
- Validate email jobs.
- Validate job application filtration.

2) *Performance Testing Objectives*: In our scope, we will provide performance testing to ensure that the system can handle at least 6 concurrent users on normal load and 10x as high load to ensure system availability and reliability. The system should maintain response time less than 1,500ms and keep users satisfactions of response time on normal and high load. Testing scalability and caching of highly requested pages and seasonal usage load.

3) *Security Testing Objectives*: In our scope, we will ensure security through testing authentication aspects. In addition, we will ensure the system is protected against SQL Injection Attacks. The system should pass all test cases, as any fault in test cases will cause significant damage to the system.

D. Test Strategy

End-to-end (System) Testing strategy and Black box methodology will be used in our test plan for different reasons such as:

- Jobs.ps has no quality assurance teams which means, the development team wants a testing strategy that makes it easy for them to test the whole system behavior without testing modules and components individually.
- System testing allows testing both functional and non-functional attributes by testing a full scenario on selected features.
- Since we have no access to the Jobs.ps code, we have to do the testing as a black box testing and the system testing built as a black box. System testing allows us to reveal the bugs and issues in the website without having the code to test integration or components individually.

We also will use agile, scrum team planning and testing as it is the most used software management in Palestine [3].

E. Test Schedule

Testing Type	Testing Process	Duration
Functional Testing	Automation	1-2 Weeks
	Manually	2-3 Weeks
Non-Functional Testing	Automation	1-2 Weeks

F. Test Resources

1) The testing team members:

- Duha Jarrar.
- Imadeddin Salama.
- Mohammad Abu Qare.

2) Testing tools to be used:

- *Jira with Zephyr Scale* as a test management tool and used in manual testing.
- *Selenium with Cucumber BDD* as an automated testing tool.
- *JMeter* as a performance and load testing tool.

Jira framework is one of the most popular frameworks to planning and testing writing and execution logging. It allows the QA Engineer to add test cases and add them to different test cycles. In addition, there are different plugins that could be connected to it such as Zephyr Scale and BDD testing that allows the user to write the automated features in BDD steps and execute the automation locally and upload the result automatically.

As Jobs.ps company is not a technical company, We tended to go forward the BDD testing using Cucumber and Selenium, so the steps of the testing are written in natural language to be understood by the company and allow them to go forward in this behavior. In addition Selenium is a free testing framework allowing UI testing with availability to high learning curve and attaching the project to regression server like Jenkins.

G. Test Deliverables

In the end of testing, we will be able to deliver the following items:

- Test Plan and Test Execution Report.
- Test cases executed and their test data.
- Bug report each with reproducing steps.
- Traceability Matrix showing each test execution connected with the feature.
- Coverage Report showing the coverage for each feature.

H. Risks and Assumptions

- Assumptions:
 - 1) The website is stable enough for testing and available.
 - 2) The website requirements are well documented and understood.
- Risks:
 - 1) Changes that occur on the website while testing may invalidate our test results.
 - 2) Mismatch between the documentation of requirement (miss ti document new requirements) and implementation.
 - 3) Unexpected issues with testing tools may affect the test schedule.

I. Dependencies

- The firewall has to be turned off during Load, Stress and Spike testing.
- The firewall has to be turned on when performing security testing.
- The testing environment must be similar to the production environment.

J. Exit Criteria

- For functional testing: Passing more than 60% of the regression tests, and resolving the critical bugs.
- For performance testing: APEX value greater than 0.85 (Good rating) in Soak and Load Tests, and greater than 0.7 in Stress and Spike tests (Fair), throughput of more than 29 transactions per second and less than 1% errors for Soak and Load Tests.
- for Security: Detecting DDos attacks and SQL injections.

K. Approvals

This test plan will use peer-to-peer approval.

L. Appendices

Appendix A, software requirement specification.

IV. TEST CASES

The First stage of our plan is creating test cases, to do that, Firstly we selected the main functionalities of Jobs.ps, and then wrote different test cases for each of them, in the following subsection we will describe the scenarios and the test cases covered.

A. Job Matcher Scenario

This feature allows job seekers to create an email notifier with the jobs that match their interests by selecting job categories, job locations, job types, and job position levels as shown in the following Fig 11. The selections could be a single selection per section or multiple selections.

Fig. 11: A screenshot of Job matcher page in Jobs.ps

The following Fig 12 shows an example of the email received by jobs.ps after using creating a matcher using feature:

In order to test the scenario, we created the following equivalence classes: One job matcher created, two job matchers created, viewing job matchers and receiving their emails. In each of the job matchers there is only selection of multiple values so we selected the data randomly and checked for multiple values. Based on this we created the test cases. We created test cases of this scenario that covered the following:

- Create a job matcher with a single value for each section (JT-T13).



Fig. 12: A screenshot of Job matcher email received from jobs.ps

- Create a job matcher with multiple values for each section (JT-T15).
- View my job matcher (JT-T14).
- Validate job matcher results with my matcher (JT-T46).

B. Apply Job Scenario

Applying for a job is the most important part of jobs.ps so, using this feature job seekers can apply for any job by filling in their information as name, email, and cover letter, and uploading their needed attachments as shown in Fig 13.

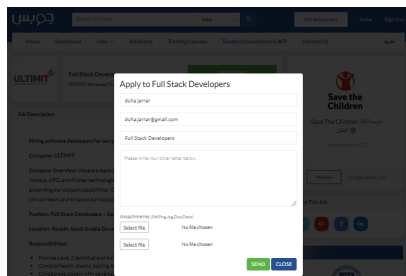


Fig. 13: A screenshot of applying for a job in jobs.ps

The test cases of this feature covered the following:

- Apply for a job with an invalid email (JT-T43).
- Apply for a job with valid data (JT-T44).
- Apply for a featured job in that week with valid data (JT-T45).

The equivalence classes are defined for each of the required data as follows:

- Valid name means its length is more than 2 chars.
- Valid email means containing '.', '@' and a length of it more than 5 chars.
- Valid attachment: its extension is one of the following: pdf, doc, docx, png, jpg.

C. Job Applications Scenario

This feature allows employers to view and do filtration on the job applications they receive from the job posts they published on jobs.ps as shown in Fig. 14.

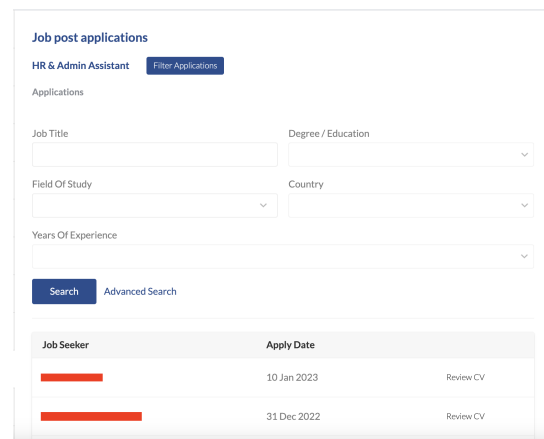


Fig. 14: A screenshot for filtering job applications in jobs.ps

This feature is covered by the following test cases:

- View applications with valid inputs (JT-T32).
- Filter applications with valid inputs (JT-T33).
- Filter applications with empty valid inputs (JT-T42).

The equivalent classes are:

- Job Title: Empty (No value), Invalid(Contains Numbers) and can be only characters.
- Degree / Education: Empty (No value) or Only valid from List.
- Field Of Study: Empty (No value) or Only valid from List.
- Country: Empty (No value) or Only valid from List.
- Years Of Experience: Empty (No value) or Only valid from List.

D. Following Scenario

This feature helps job seekers get employment in their dream companies by allowing them to follow job providers/companies, the following Fig. 15 shows the following list page.

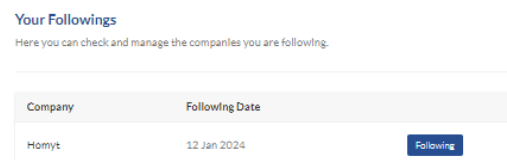
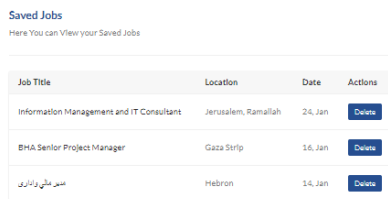


Fig. 15: A screenshot of the Following page in jobs.ps

This feature includes 1 test case, which is "Validating follow job provider" functionality (JT-T20).

E. Saved Jobs Scenario

This feature helps job seekers to save any job they are interested in to apply for them later for example, the following Fig 16 shows the saved list page.



Job Title	Location	Date	Actions
Information Management and IT Consultant	Jerusalem, Ramallah	24, Jan	Delete
BMA Senior Project Manager	Gaza Strip	16, Jan	Delete
مدير مالي وإداري	Hebron	14, Jan	Delete

Fig. 16: A screenshot of the Saved page in jobs.ps

This feature includes 1 test case, which is "Validating save job" functionality (JT-T5).

F. Email Jobs Scenario

This feature allows anyone to share any job with others by sending it to their emails, this requires filling in the sender name and the receiver's email as shown in the following Fig 17.

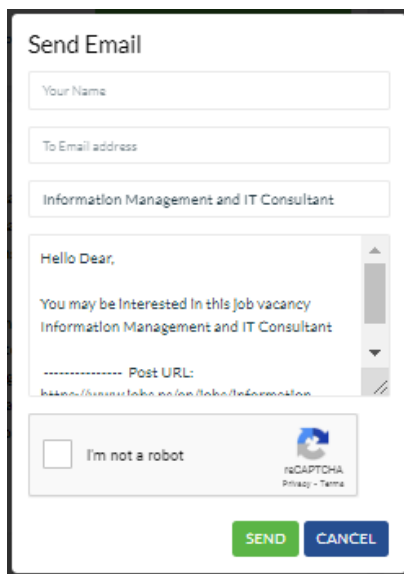


Fig. 17: A screenshot of email job form in jobs.ps

This feature is covered by the following test cases:

- Email job with invalid receiver email (JT-T21).
- Email job with invalid sender name (JT-T39).
- Email job with empty sender name and receiver email (JT-T40).
- Email job with valid sender name and receiver email (JT-T41).

The equivalence classes are defined for each of the required data as follows:

- Valid name means its length is more than 2 chars.
- Valid email means containing '.', '@' and a length of it more than 5 chars.

G. Browse Jobs By Location Scenario

This feature allows job seekers to browse jobs by locations such as cities, outside Palestine jobs, and remote jobs. The following Fig 18 shows locations options.

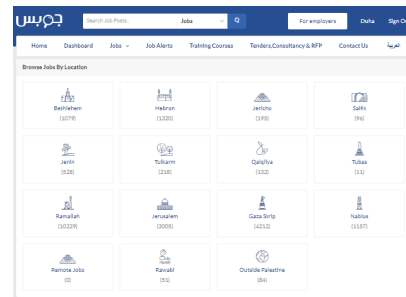


Fig. 18: A screenshot of browse jobs by location page in jobs.ps

There are different equivalence classes in the selection for job browse by locations. There are different cities in Palestine class, Remote Jobs and Outside of Palestine jobs. We created test cases for each. This feature is covered by the following test cases:

- Browse for jobs in a specific city (JT-T16).
- Browse for jobs outside Palestine (JT-T17).
- Browse for jobs in remote jobs (JT-T19).

H. Resume Builder Personal Information Scenario

In this feature, the job seeker is allowed to insert the personal information for him to be saved in jobs.ps and be used for applying jobs and building a resume as PDF file, check the figure 19 showing Resume Builder feature tab information.

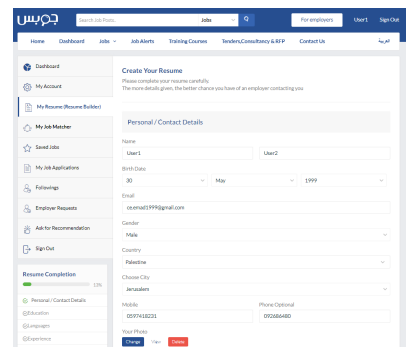


Fig. 19: A screenshot of Resume builder form in jobs.ps

The personal Information saved has many different fields such as: first name, last name, email, mobile, phone number, day of birth, month of birth, year of birth, social media facebook optional link, social media optional linkedin link, country and city. For such information we created different equivalence classes. First and Last name can be either be: Empty (No value), Invalid(Contains Numbers) and can be only characters. Thus, we created 3 test cases for such:

- Add User Personal Information by filling all valid details (JT-T6).

- Add Personal Information for User with Invalid First name and Last Name (JT-T9).
- Add Personal Information for user with Empty First name and Last name (JT-T10).

For the email. There are also 3 equivalence classes: Empty (No value), Invalid(Doesn't match email) and can be valid email. Thus, we created 2 test cases for such:

- Add User Personal Information by filling all valid details (JT-T6).
- Add Personal Information for User with Invalid Email (JT-T7). This test cases contains test data to be empty or invalid.

We also covered the validation for phone numbers which contains different equivalence classes: Empty (No value), Valid (10 numbers for mobile, 9 numbers for phone), not valid (more or less than 10 numbers for mobile and more or less than 9 numbers for phone) or can be invalid as contains characters. As the report asked to have only one test covered for such scenario, we were able to cover these test cases:

- Add User Personal Information by filling all valid details (JT-T6).
- Add Personal Information for User with Invalid Mobile and Phone Number (JT-T8). Where we checked if it was invalid values with less numbers than expected.

The previously mentioned test cases were executed in Automation Testing part to be in regression, as the resume building information will be used widely in jobs.ps before applying to a job by a job seeker would occur. These cases represent regression to ensure any changes and information could be added to resume builder doesn't effect the personal information.

There are also facebook and social media links that were tested against such invalid facebook and linkedin URLs. There are different equivalence classes: Empty (Allowed as it is optional), Invalid (base URL is not facebook or linkedin), valid values. As the report asked to have only one test covered for such scenario, we were able to cover these test cases:

- Add User Personal Information by filling all valid details (JT-T6).
- Add Personal Information for User with Invalid Social links (JT-T12). The data contained different host URLs instead of facebook and linkedin.

Finally, we also covered updating the image for the job seeker user. There was a test case of changing of the user to create a PDF resume file covered through the test case: Update photo in Personal Information(JT-T11).

1. Search Jobs Scenario

This feature allows job seekers to search for any job using 3 ways as shown in Fig 20.

1) *Search in Companies:* This feature allows job seekers to search for any company.

This feature is covered by the following test cases:

- Search companies in jobs with a valid company name (JT-T29).

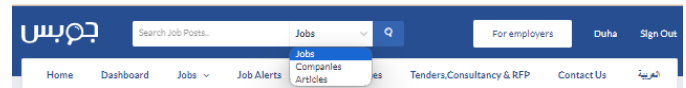


Fig. 20: A screenshot of search in jobs.ps

- Search companies that are not listed company names (JT-T30).
- Search companies in jobs with multiple valid company names (JT-T35).

2) *Search in Articles:* This feature allows job seekers to search for any article. This feature is covered by the following test cases:

- Search article with existing articles (JT-T31).
- Search article with a sentence (JT-T36).
- Search article with random text (JT-T37).

3) *Search in Job posts:* This feature allows job seekers to search for any job post. This feature is covered by the following test cases:

- Search job posts in jobs with empty text (JT-T23).
- Search job posts in jobs with invalid text with no job posts (JT-T24).
- Search job posts in jobs with a short text (JT-T25).
- Search job posts in jobs with misspelled keywords (JT-T26).
- Search job posts in jobs with elastic search (JT-T27).
- Search job posts in jobs with a very large text (JT-T28).
- Search job posts in jobs against sql injection (JT-T34).
- Search for correct values (JT-T38).

Search equivalence classes, as it is a free text field with some constrains: Invalid (Large Text input, Empty text, less than 3 characters, Sql injection, misspelled keywords), Valid (More than 3 characters, short sentence, multiple keywords).

All those test cases are documented in the Zephyr Scale in Jira with their information as shown in Fig 21.

Fig. 21: Some of the documented test cases on Zephyr

Each test case was documented in detail including the Test case name, Objective, Precondition, Priority, Estimated time, Labels, and Steps, Expected result, and Test data as shown in Fig 22.

V. TEST CASE RESULTS

The second stage of our plan is executing the test cases and reporting bugs if exists, we did that using JIRA Automated Management tool. We created a project for jobs.ps in order to

Fig. 22: Documented test case in details

report all the information and automate the generated report by the tool itself. In addition we used Zypher Scale for adding test cases and test executions.

We added all the functional requirements to the system by dividing them into Epics containing the highest level features. Each EPIC is divided into different stories representing the functional requirements should be added by such story. As there are no agile development in this project, we didn't add any sub-tasks for each story, check figure 23. These epics and stories were created to keep track of all features in the system, and build test cases and connect each of the cycles, test cases, bugs opened to the issues created.

Fig. 23: Jira EPICs and Stories.

Then, we created different test cases based on the features selected to be tested and explained in the previous section. The test cases are first created by adding a name for the test case and connect them to the story it is mentioned for which is the feature we want to test, check Fig. 24.

Then, we created test execution cycles, A test execution for Manual test cases and an automated test execution cycles. The Automated test execution cycle can be executed by uploading

[illegible]

the results through cucumber framework with connecting test cases with same feature test case key.

The test cases connected to Manual Test Execution cycle were inserted as test scripts as steps. These steps contains the information to reproduce the test case in case a failure happened. The last steps contain the expected behavior of the system and the test data to be inserted in such step. For the automated test cases, the test script was Gerklin BDD containing the behavior development driven testing approach by writing the steps the same as feature file in the automation testing project.

The screenshot shows the 'Test Cycle' page with a table of test cycles. The table has columns for Name, Status, and Progress. Two test cycles are listed: 'JF 412 Automated Test Cases Cycle' and 'JF 413 Manual End to End Test Cycle'. Both show a progress bar at 100% and a status of 'Completed'.

Name	Status	Progress
JF 412 Automated Test Cases Cycle	Completed	100%
JF 413 Manual End to End Test Cycle	Completed	100%

After creating the test cycles we can simply add the needed test cases to the test cycle we want. The tool allows us to create different test cycles which is a very helpful feature in case of agile development and testing features. We only attached the manual test cases to the Manual test cycle for manual execution while other automation test cases we moved it into Automation test cycle. Check Fig. 26 showing the manual test cycle execution.

[illegible]

After executing the test cases we were able to create the bug tickets in each test case and include the bug into the story related to the bug opened, check Fig. 27 showing a bug opened and connected with the story for it.

In both test executions, we executed 42 test cases: 34 test cases were manual and 9 test cases were automated test cases. 7 automated test cases out of 9 passed and 2 automated test cases out of 9 failed. 19 manual test cases out of 34 passed and 15 manual test cases out of 34 failed.

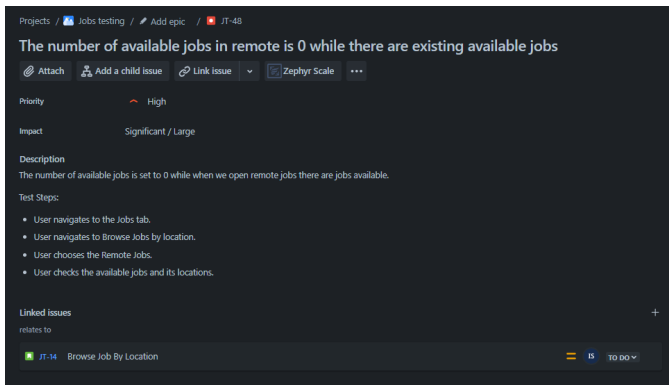


Fig. 27: Bug opened Related to Story.

The failed 17 test cases resulted in 22 opened bugs. Some of the test cases resulted in 1 opened issue and some of them resulted in more issues to be opened for such issue. Each of the bugs tickets were connected to the related stories.

The figure 28 shows the traceability matrix report created by the Jira tool. The traceability matrix connects the test cases with the feature and the test case with all related bugs created by each test case.

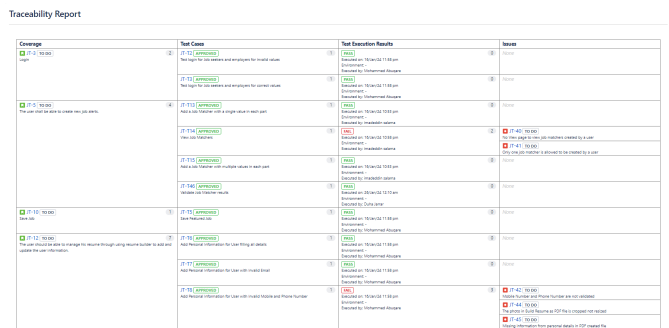


Fig. 28: Traceability Matrix Report by Jira tool.

The bugs reported were in different areas, such as: input validation, missing features than requirements, errors in results, invalid search operations and many others. All of the bugs and related test cases are mentioned in Appendix ??

VI. TEST CASES ANALYSIS

After test cases execution we used Jira Dashboard to analyze the results and issues after the execution.

We were able to execute 42 test cases in total. 34 manual test cases and 9 automated test cases. These were distributed by the execution type manual execution cycle and automated execution cycle, check Fig. 29.

In these test cycle executions, we were able to report 20 bugs in total. 4 bugs were reported by automated test execution and 16 bugs were reported by manual test execution cycle. These bugs differs in their priority as the figure 30 shows. The bugs were in 3 categories of priority: 9 are medium priority as they contain input validation. 7 are high bugs contain core functionality errors. Finally, 4 of them are very high priority.



Fig. 29: Test Case Execution.

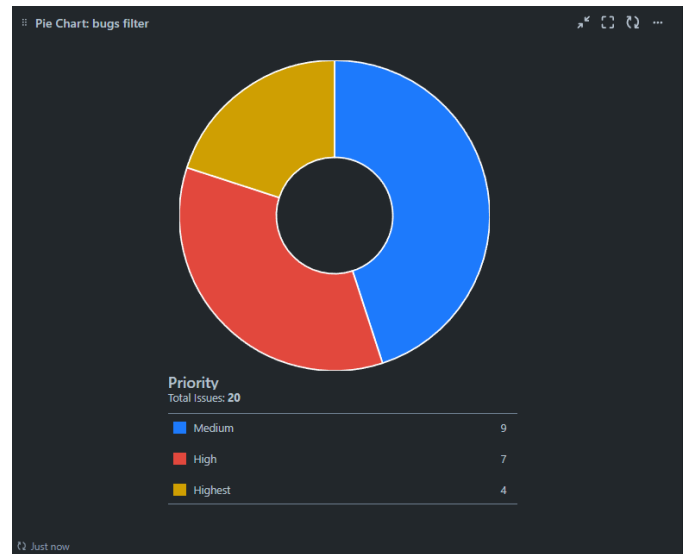


Fig. 30: Chart Pie of bugs per priority.

Fig. 31, shows the test case execution per each story coverage. The figure shows each of the stories related to it. This graph is reported by Jira tool. The figure shows that the highest number of tests were executed to Search jobs feature as it represent one of the main functionalities of the website. It was followed by emailing job and Building Resume features which are features expected to be highly used in the website to apply for jobs. And the last one is following companies and view job applications as it should be similar for any job application or any company and can be checked by least number of tests.

VII. TEST AUTOMATION

As part of the project, we created different automation tests to be part of this project. We selected Cucumber Behavior Driven Development framework to be the main testing framework with the usage of Selenium framework to implement the test cases defined by cucumber feature files.

As the Fig. 32 shows, the Automation part runs through specific flow. The flow starts by writing BDD Feature file to Jira execution steps and exporting it as a feature file. The file

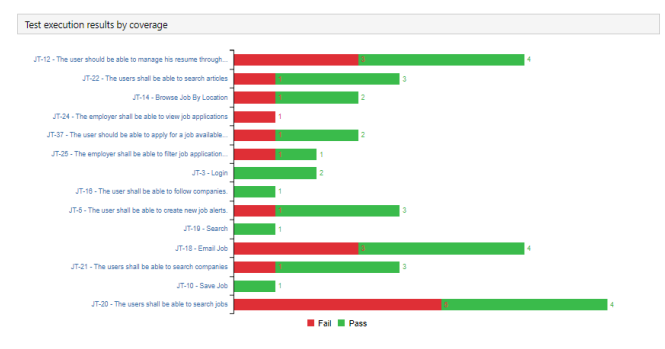


Fig. 31: Test Execution Coverage Per Feature.

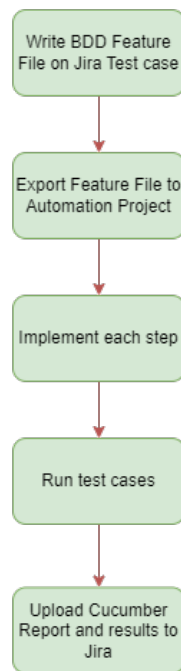


Fig. 32: Automation Sequence Flow

is added to the automation project features. Each of the steps either be reused or implemented in the project step definitions. Then we execute the automated test cases and the execution generate a report. The report is uploaded through a script to Jira automation cycle.

The Cucumber framework works mainly using Maven and by adding Cucumber Plugin into the IDE. The main step of creating different automated test cases is to create the feature files. The feature files represent files that contains the steps of running the test. Cucumber BDD framework grants high reusability among the test cases, in such we are able to reuse different steps from existing tests such as the login operation. While the login operation can be a test case itself. It is a part of other test cases containing user account interaction such as filling Resume Builder personal information.

According to the figures 33 and 34, we can see that the steps that are used can be exist in another feature. This way

```

1 Feature: Test login for Job seekers and employers
2
3 @TestCaseKey=JT-T2
4 Scenario Outline: Test login for Job seekers and employers
5   Given Navigate to "<login>" Page
6   When Enter Email "<username>"
7   And Enter Password "<password>"
8   And Click on Login Button
9   Then It should display "<successfully>"  abugare3, 1/11/2024 1:54 AM • Init
10
11 Examples:
12 | login | username | password | successfully |
13 | job-seeker/login | abugare | xxxxxxxxxx | Email address is invalid email |
14 | job-seeker/login | abugare@gmail.com | xxxxxxxxxx | Invalid Credential |
15 | job-seeker/login | abugare | xxxxxxxxxx | Email address is invalid email |
16 | job-seeker/login | | | | Email address is invalid email |
17 | employer/login | abugare@gmail.com | xxx | Maximum Password length is 4 |
18 | employer/login | abugare@gmail.com | xxxxxxxxxx | Invalid Credentials |
19 | employer/login | abugare | xxxxxxxxxx | Email address is invalid email |
20 | employer/login | | | | Email address is invalid email |
21
  
```

Fig. 33: Login Feature File.

```

1 Feature: Save Featured Job
2
3 @TestCaseKey=JT-T5
4 Scenario Outline: Save Featured Job
5
6   Given Navigate to "<login>" Page
7   When Enter Email "<username>"
8   And Enter Password "<password>"
9   And Click on Login Button
10  And Navigate to Home Page
11  And Click on featured job
12  And Click on save job
13  Then Should display saved successfully
14
15 Examples:
16 | login | username | password |
17 | job-seeker/login | duha.jarran@gmail.com | D123456j |
  
```

Fig. 34: Login Feature File.

we increase the reusability in our testing project with the same functions used (no need to re-implement it again).

Each step of these features are transferred into Java implementation through adding Cucumber annotation based on the step definition in features file. For example if the step starts with Given, the implementation method should be annotated by @Given with the same step definition message to be automatically transferred for execution. When each step is defined we can run the class Runner to run the features inside a specific folder. Cucumber Runner searches through the project for the step definition in Java language and run the function based on it by passing the parameters defined through examples. This way we can run different examples or test cases with the same feature file, check figure 35.

```

Examples:
| login | username | password | navigate_to |
| job-seeker/login | abugare@gmail.com | Al | job-seeker |
| employer/login | abugare@gmail.com | Al | employer/home |
  
```

Fig. 35: Feature File Examples.

As selenium is based on creating a web driver object and use it to move between pages and detect elements and fill these elements with input texts. We were not able to use different

web drivers in different classes of different methods. To enhance the reusability we needed to implement one web driver for all tests that can be created once and used everywhere. Hence, we created a Singleton class called DriverManager, which is a class contains an object of web driver that creates it only once and use it everywhere. This way we were able to reuse different methods in different classes with no need of re implementing such methods like login logic, check figure 36 for Singleton Implementation.

```

1 package com.core;
2
3 > import ...
4
5 24 usages 1 ImadSalama +1
6
7 public class DriverManager { ImadSalama, 1/13/2024 12:02 PM • Refactoring Project Code
8
9     1 usage
10     private static final int WAIT_FOR_RESULTS = 15;
11
12     8 usages
13     private static WebDriver webDriver;
14     no usages 1 ImadSalama
15     private DriverManager(){
16     }
17
18     4 usages 1 ImadSalama
19     public static WebDriver getInstance(){
20         if (webDriver == null){
21             WebDriverManager.chromedriver().setup();
22             webDriver = new ChromeDriver();
23             webDriver.manage().window().maximize();
24         }
25         return webDriver;
26     }
27
28     8 usages 1 ImadSalama
29     public static void closeDriver(){
30         webDriver.close();
31         webDriver.quit();
32         webDriver = null;
33     }
34

```

Fig. 36: WebDriver Manager Singleton Code.

Each of the test cases in the feature file is connected per TestCaseKey annotation. This annotation connects the test case key of the feature file with the test case in Jira for automation upload for the results report. After the test cases are executed. Cucumber generated a report of the test cases and their results. It shows if the test case pass per each step and if any of the test case failed it shows the step and exception that it failed and description of the issue appeared so the Test Engineer can easily detect the issues or catch the test issue to be solved, check figure 37.

In the Cucumber Project, we defined different packages:

- **Constants Package:** The package contains the constants classes that we used across different classes and test cases. The constants contains classes like: Elements locators to locate different elements such as username in login, password in login ...etc. In addition to Regex Matchers to match some elements and digits for further detection. Website URLs to move between pages instead of using buttons to move between pages. Finally, the error locators to locate error messages when we expect errors.
- **Core Package:** The core package contains the DriverManager class which is the class managing the singleton web driver element to be used across different test cases step definition.

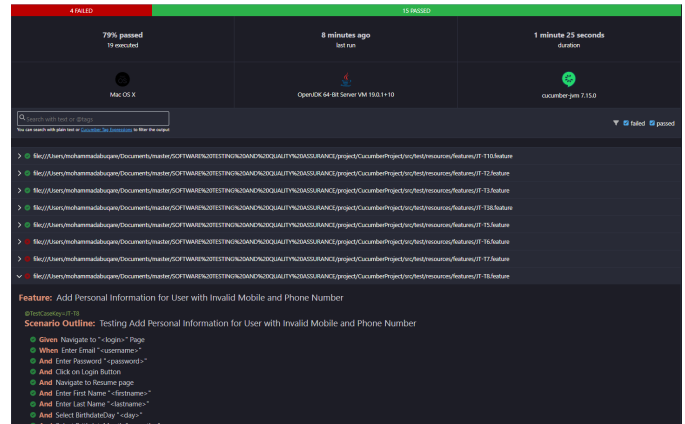


Fig. 37: Cucumber HTML Report.

- **stepDefinition Package:** The step definition package contains information about the definition steps of annotated steps inside feature files. It was grouped together based on the assignment required. For example the Login-Feature steps was created as a separate class to test login and then it was used by other features such as Save Jobs and Resume Builder. The ResumeBuilderFeature class contains all steps related to filling resume builder test cases. SaveJobFeature contains all steps related to save jobs feature. Finally, Search Feature contains all steps used for searching for jobs, articles, companies...etc.

The locators we used in our Testing project could be either XPath locator. or CSS locators. The CSS Locator represents locating different elements through HTML and CSS attributes such as title, name, type ...etc. XPATH locator represents locating an element through traversing nodes of HTML tags from parent to children nodes with different attributes. To detect a locator we either used a very simple method such as locating an input field with title firstname, which is very unique and can't be changing when changing the design. Otherwise for complex locators such as error elements it was required to use XPATH locator to locate different HTML tags to reach the error tag and check if it is displayed.

Finally, after we run the project as Cucumber Project, we get the report added in reports folder inside the project. We don't need to report the results manually to Jira. We can use a script to load this report and the report will reflect the test case execution result by adding a new test execution cycle. This script uses a key from JIRA and compress the reports in .zip file and then uploads the result using HTTP request, check figure 38 showing the script code for uploading the report.

The source code of the project is available through github repository through the link in the references [7]. We conducted Similar to real work experience environment managing the code by doing Peer-To-Peer Review of the pull requests for each Pull Request opened.

In our Test Automation Part, we covered different test cases in the system to be automated test cases. Which are: JT-T2, JT-T3, JT-T5, JT-T6, JT-T7, JT-T8, JT-T9, JT-T10, JT-T38 test

Fig. 41: Running headless Jmeter mode using Command line

Elapsed time	Number of active threads
00:00:00	0
00:00:00.001	6
00:04:59.999	6
00:05:00.000	0

Fig. 42: Soak Load Test

2) *Load Test*: The load test should handle 60 active users threads over a period of about 5 minutes to apply constant load to the website illustrated in Fig. 43.

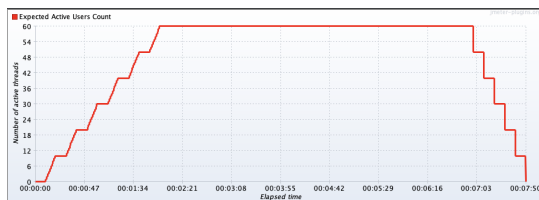


Fig. 43: Load Test

3) *Stress Test*: The stress test should handle a gradual increase in the number of active users by starting from 20 active users up to 200 active users over a period of time (about 4 minutes) as illustrated in Fig. 44.

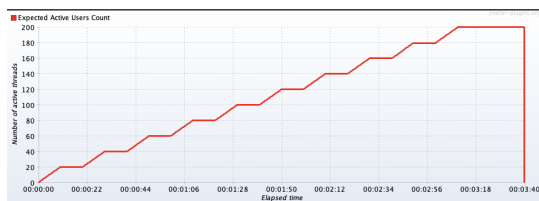


Fig. 44: Stress Test

4) *Spike Test*: The spike test should handle a sudden increase to 400 active users in a very short time (1 minute or less) as illustrated in 45.

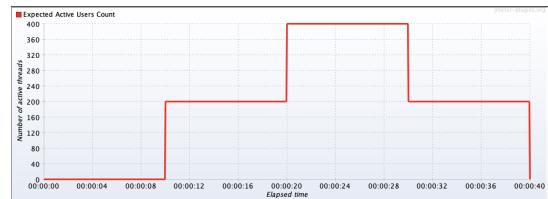


Fig. 45: Spike Test

The Table I, shows the coverage matrix between nonfunctional requirements and performance test types, which gives us a clear picture of performance tests coverage against the nonfunctional requirements and performance metrics.

NFR	Soak Test	Load Test	Stress Test	Spike Test
NFR1.0	No	Yes	Yes	No
NFR2.0	Yes	Yes	Yes	No
NFR3.0	No	Yes	Yes	Yes
NFR6.0	No	No	Yes	Yes

TABLE I: Test and NFR Matrix Coverage

As shown in I, all the nonfunctional requirements that are related to performance have been addressed.

The Table II summaries the results of all tests, it clearly shows that the current implementation/infrastructure of the websites failed to handle Load, Stress and Spike tests and only passed Soak test.

Metric	Soak Test	Load Test	Stress Test	Spike Test
Active Users	6	60	200	400
Duration	5m	7:50m	3:40m	40s
Error Rate	0%	0.1%	95.28 %	97.35%
Server Down	No	No	Yes	Yes
Failure at	None	None	150 AUs	150 AUs
Apdex	0.442	0.030	0.001	0.00
RT (AVG)	1508.98ms	11532.45ms	1050.82ms	782.70ms
RT (90th pct)	3237.0ms	19229.80ms	142.00ms	702.00ms
Throughput (t/s)	3.96	4.34	103.18	233.04

TABLE II: Performance Tests Summary

The Stress and Spike tests caused the website to go under maintenance as shown in Fig. 46, after that the gateway started responding to the performance test script with errors.

Early in the tests after few attempts of checking the performance of the websites, Jobs.ps has enabled Cloud-Front firewall which blocked us (forbidden access) from continuing the test in which it answers the NFR6.0 related to DDos security attacks as shown in Fig. 47.

F. Analysis

On this section, we will analyze the results of the three performance testing types using the generated Jmeter reports.



Fig. 46: Server Down

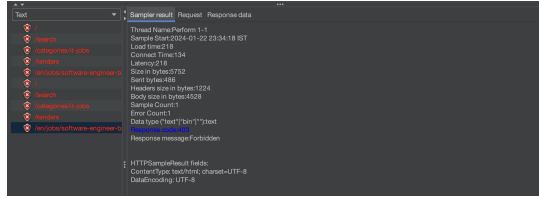


Fig. 47: Blocked by Firewall - Forbidden Access

1) **APDEX (Application Performance Index)**: APDEX measures the satisfaction of users with the web application response time. Where it depends on two factors the Tolerating (T) and Frustration (F) factors. The values that are less than T are considered satisfied, values that are between T and F are tolerating, values that are above F are frustrations according to the industry standard. The T and F are defined as 500ms and 1,500ms, respectively. Apdex ranges from 0 to 1, which ranges from totally frustrated to fully satisfied, respectively.

The APEX equation is calculated according to [8] as follows:

$$Apdex = (SatisfiedRuns + ToleratingRuns / 2) / TotalRuns$$

The result of the equation determines the users satisfactions according to Table III, the performance rating according to [8].

Apdex Rating	Apdex
Excellent	0.94–1.00
Good	0.85–0.93
Fair	0.70–0.84
Poor	0.50–0.69
Unacceptable	0.00–0.49

TABLE III: Apdex Ratings [8]

Based on the definition of Apdex and according to the summary in Table II, all users of all performance tests are disappointed with **unacceptable** rating score in highlight of Apdex Rating Table III.

The APEX values of all tests were less than 0.49. Even, the Soak test's APEX result was (0.44) which is considered normal load.

2) **Response Time**: Looking at the response time of all tests as shown in Table II, the 90th percentile of the Soak test is 3237.0 and the average is 1508.98ms response time, which

means that 90% of users got response time less than 3.2s and the average is 1.5s, which is above 1.5s in most cases, which is classified as bad response time.

In the load test, the average response time is about 11.5s and the 90th percentile is about 19.3s, which is a severely bad response time.

Stress Test shows that the average response time is 10.5s on average but the 90th percentile response time shows that 90% of which have received responses in less than 0.14s which indicates an instability of sever response time, the reason for that the server went down and the gateway response were very fast, responding with 502 error message, Fig 48 shows the distribution of responses time over number of active threads, where it clearly shows that response time becomes very faster after 150 active threads when server went down.

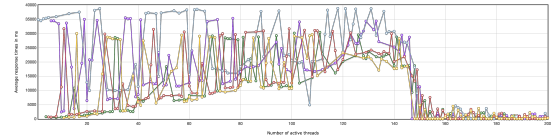


Fig. 48: Active Users vs Response Time (ms) - Stress Test

The Spike Test looks stable in terms of the 90th percentile and average response time since both were very close (702 ms and 782.7 ms, respectively) but that is due to the fact that the server went down very quickly.

3) **Throughput**: Throughput is used to measure the number of transactions that the server website can handle in seconds. Regarding the soak test and the load test, the website was handling about 4 transactions per second, which is very far from 29 transactions under normal conditions that was calculated early in the metrics section. In Stress Test and Spike Test, the throughput was increased dramatically to 103 and 233 transactions per seconds, respectively, but that is due to gateway fast response of errors again. This is also clearly shown in Fig. 49, where the red line represents the transactions with errors.

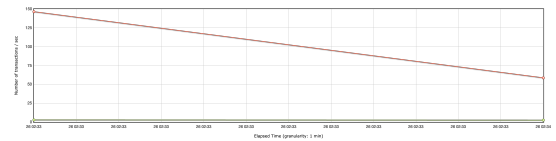


Fig. 49: Total Transactions Per Second - Spike Test

4) **Errors**: There were mainly two error types that appeared during the load, stress and spit test that are due to server down and inability to handle requests as shown in Fig. 50.

Type of error	Number of errors	% in errors	% in all samples
502/Bad Gateway	12315	99.99%	97.34%
Non HTTP response code: org.apache.http.NoHttpResponseException/Non HTTP response message: www.jobs.ps:443 failed to respond	1	0.01%	0.01%

Fig. 50: Gateway Errors - Spike Test

IX. QUALITY ENHANCEMENT TIPS

While we were testing jobs.ps, we took some notices that could help users experience jobs.ps and improve the product quality such as:

- Using more clear labels as in:
 - Email job: when you read it you understand it sends email for a job but its functionality reflects its sharing a job with someone using email.
 - Search job by location or other ways as shown in Fig 51 in the red box, the "Search" word should be replaced by "Browse" to be more accurate.

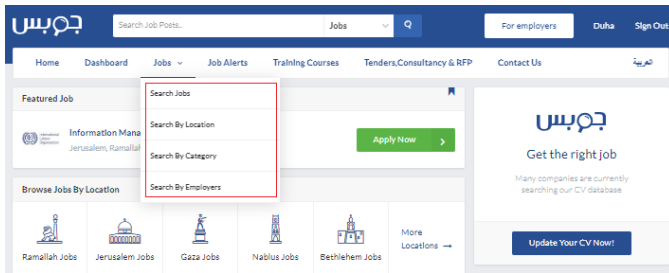


Fig. 51: Search section of jobs.ps

- In applying for a job, it's not clear if the Resume/CV will be attached automatically or if there is a need to attach it since it should exist in the user profile.
- Adding archive job offers for the jobs with expired dates.

X. CONCLUSION

This reports conducts a testing experiment on Jobs.ps website in functional and performance functionalities. The reports takes into consideration different core functionalities in the system to be tested as the website is used widely in Palestine to offer jobs by job providers and apply for jobs by job seekers. We tested different functionalities such as job search, job locations, apply for the job, job matchers, and many others. We were able to detect 20 bugs in the system in functional requirements and generate a traceability matrix reported by Jira. In addition, we conducted automation testing using BDD with Selenium in Java. The automation contained the test cases that are widely used in the system and need continuous regression. The automation tests were able to discover 4 bugs by itself. We also conducted a performance experiment to test the load and performance of the applications using JMeter tool to conduct Soak Load, Load, Stress and Spike testing for the website. Based on Apdex metrics the result was 0.44 in best of normal load which represents the user response time satisfaction, which is unacceptable rating. The experiments we conducted indicates a reflection that the local Palestinian companies do not fully test the applications, as the scenarios we took into consideration were essential and should be all valid cases, but 20 bugs were produced from the test cases. Further work should be done by applying the same experiments over several local websites in order to have solid reflections on the local market software quality.

REFERENCES

- [1] "SimilarWeb PRO," https://pro.similarweb.com/#/digitalsuite/websiteanalysis/overview/website-performance/*999/3m?webSource=Total&key=jobs.ps, note = Accessed: 2024-01-27.
- [2] "About Us - Jobs.ps," <https://www.jobs.ps/en/about-us>, note = Accessed: 2024-01-27.
- [3] "State of Agile Report 2022," <https://info.digital.ai/rs/981-LQX-968/images/AR-SA-2022-16th-Annual-State-Of-Agile-Report.pdf>, note = Accessed: 2024-01-24.
- [4] B. Abdalmaq et al., "Introducing agile software development methodology (scrum) into a software development project at a local firm," Ph.D. dissertation, 2013.
- [5] "Test Cases — Zephyr Scale Cloud Documentation," <https://support.smartbear.com/zephyr-scale-cloud/docs/index.html>, note = Accessed: 2024-01-24.
- [6] "Home - Apache JMeter - Apache Software Foundation," <https://cwiki.apache.org/confluence/display/JMETER/Home>, note = Accessed: 2024-01-27.
- [7] "Github Repository — Cucumber Test Project Source Code," <https://github.com/abugare3/CucumberProject>, note = Accessed: 2024-01-24.
- [8] "Apdex — AlertSite Documentation," <https://support.smartbear.com/alertsite/docs/monitors/metrics/apdex.html>, note = Accessed: 2024-01-27.

APPENDIX A

FUNCTIONAL REQUIREMENT

This appendix contains requirement specification tables per category.

TABLE IV: Home: Functional Requirements

User	System	Description
UR1.0		The user shall be able to upload and update his CV.
	SR1.0.1	The system shall allow the user to upload his CV as a PDF file with maximum file size of 10 MB.
	SR1.0.2	The system shall allow the user to edit his CV by uploading a new file with maximum size 10 MB as a PDF file.
UR1.1		The user shall be able to view Today's jobs and latest Tenders / RFPs / Consultancy
	SR1.1.1	The system shall show the user Last 7 Days jobs by their basic information job title and the location with a forwarding link to the job description page.
	SR1.1.2	The system shall show the user the latest tenders and consultancies available on the system by their basic information (tender title, location, deadline) with a forwarding link to the tender information.
UR1.2		The user shall be able to subscribe to job notifications through email.
	SR1.2.1	The system shall allow the user to subscribe for new jobs through email notifications
	SR1.2.2	The system shall allow the user to provide his email to subscribe for email notifications for the newest jobs.
UR1.3		The user shall be able to view the featured job and its information (provider name, image, title, location).
	SR1.3.1	The system shall allow the user to view the featured job with its information (provider name, image, title, location).
	SR1.3.2	The system shall forward the user to the job description page and apply for the job page if he selects the job to view it.

TABLE V: Dashboard: Functional Requirements

UR2.0		The user shall be able to create new job alerts.
	SR2.0.1	The system shall allow the user to create new job alerts.
	SR2.0.2	The system shall forward the user to the job alerts page to add a new alert and manage created alerts.
UR2.1		The user shall be able to search among opening jobs, through the search phrase, location, category and job type.
	SR2.1.1	The system shall allow the user to search among opening jobs, through the search phrase, location, category and job type.
	SR2.1.2	The system shall allow the user to filter the jobs based on the filters of these jobs including search phrase, location, category and job type.
	SR2.1.3	The system shall order the viewed jobs based on the publish date from the provider.
	SR2.1.4	The system shall show the user each job title, location, provider and published date.
UR2.2		The user shall be able to view his saved jobs in the system.
	SR2.2.1	The system shall allow the user to view his saved jobs in the systems with each: information job title, location, provider and published date.
	SR2.2.2	The system shall allow the user to delete a job from his saved jobs in the system.
	SR2.2.3	The system shall allow the user to view the job in saved jobs until the job is removed by the job provider.
UR2.3		The user shall be able to view his job application on each job he sent an application to.
	SR2.3.1	The system shall allow the user to view all job applications sent to job providers with its information (job application date, status, job information, CV).
	SR2.3.2	The system shall be able to view these applications on employer side for the employers to view the users who sent job applications and update their status.
	UR2.4	The user shall be able to update his account basic information (name, phone number, CV attached, password, education, skills, work experience, projects).
	SR2.4.1	The system shall allow the user to view his basic information like (name, phone number, CV attached, password, education, skills, work experience, projects).
	SR2.4.2	The system shall allow the user to update his information and track a record of update date and old data and new data after the update.
UR2.5		The user shall be able to upload and update his CV.
	SR2.5.1	The system shall allow the user to upload his CV as a PDF file with maximum file size of 10 MB.
	SR2.5.2	The system shall allow the user to edit his CV by uploading a new file with maximum size 10 MB as a PDF file.

TABLE VI: Jobs: Functional Requirements

UR3.0	The user shall be able to view the jobs available on the website based on different categories.
SR3.0.1	The user shall be able to view jobs by different categories: cities, job type (full time, part time, both full time and part time, contracts and consults) and fields.
SR3.0.2	for each category, the system shall show the available options based on the number of jobs available for each field descendingly.
SR3.0.3	The system shall show an option for the user to check the most recent published jobs.
SR3.0.4	The system shall allow the user to search for a job based on the search phrase and the cities that offer the job.
UR3.1	The user shall be able to view the available jobs in a city.
SR3.1.1	The system shall show the user the available jobs in the selected city, each job name, job provider, city and the publish date.
SR3.1.2	The system shall allow the user to select a job of the provided jobs.
SR3.1.3	The system shall show the user all job information when the user selects a job, job information include: Job Description, Job Requirements, application deadline, job location, job title, job type, position level, salary, degree, experience in years, category and application instructions.
SR3.1.4	The system shall allow the user to email the job provider by forwarding him to the email with the receiver email to be the job provider.
SR3.1.5	The system shall show different jobs from the same provider as the latest jobs by this job provider.
UR3.2	The user shall be able to view the available jobs by category.
SR3.2.1	The system shall show the user all available jobs in a selected category.
SR3.2.2	The system shall allow the user to filter these jobs inside a selected category by job title, locations, position level, job type, category.
UR3.3	The user shall be able to view the available jobs by job types (full time, part time, both full time and part time, contracts):
SR3.3.1	The system shall show the user each of these job types so the user can choose between them.
SR3.3.2	The system shall show the available jobs based on the selected job type.
UR3.4	The user should be able to save a job and manage saved jobs.
SR3.4.1	The system should allow the user to save a job through the job description page.
SR3.4.2	The system should allow the user to show his saved jobs through the saved jobs settings. Each saved job should be listed with its information like job title, location, date with a forwarding link to the job description.
SR3.4.3	The system should allow the user to remove a saved job from the saved jobs settings.
UR3.5	The user should be able to share the job through social media websites.
SR3.5.1	The system should allow the user to share his jobs on different social media platforms (Twitter, Google+, LinkedIn, Facebook).
SR3.5.2	The system should forward the user on a different pop-up page to share the feed page on the social media platforms and add an automatic message containing "job title- job link" on the feed space and allow him to post it.
SR3.5.3	The system will have no access to the social media platforms as it only transfers the automated message and the authentication of the social media should be done on the pop-up page by the user himself.
UR3.6	The user should be able to apply for a job available on the system.
SR3.6.1	The system should prevent the user from applying to a job if his CV completion is less than 50%.
SR3.6.2	The system should allow the user to apply for a job by the job description page.
SR3.6.3	The system should allow the user to write a cover letter and upload up to 2 attachments with the apply request.
SR3.6.4	The system should send this request to the job provider account as a job applicant and add all his CV data, cover letter and the attachments on the job applicant request.
UR3.7	The user should be able to view the featured job on the website.
SR3.7.1	The system should show the featured job on top of the homepage marked as featured job.
SR3.7.2	Once the feature job is selected, the system should allow the user to view all the job related information (Job Description, Job Requirements, application deadline, job location, job title, job type, position level, salary, degree, experience in years, category and application instructions) which is the information shown for each job by the user.
SR3.7.3	The system should allow the admin to choose the featured job on the system to be shown to users in the homepage with a duration that the admin set.

TABLE VII: Account: Functional Requirements

UR4.0	The user shall be able to change the password and preferred language.
SR4.0.1	The system shall allow the user to change his password by providing a password and confirmation on the password.
SR4.0.2	The system shall allow the user to change his language preferences choosing between English and Arabic languages.

TABLE VIII: Follow: Functional Requirements

UR5.0	The user shall be able to follow companies.
SR5.0.1	The system shall allow the user to follow a company if the user entered their page.
SR5.0.2	The system shall send the user notifications and emails if the company created any new job post.
SR5.0.3	The system shall allow the user to remove any companies from followings.

TABLE IX: Job Matcher: Functional Requirements

UR6.0	The user shall be able to manage his job matcher.
SR6.0.1	The system shall allow the user to create a new job matcher by adding a job category, job location, job type and position level as the user can choose many options for each job matcher.
SR6.0.2	The system shall allow the user to view, update, delete his job matcher.

TABLE X: Ask for Recommendation: Functional Requirements

UR7.0	The user shall be able to ask for recommendation on a reviewer email.
SR7.0.1	The system shall allow the user to send recommendation requests for a reviewer on their email by entering his/her name, email and automated message.
SR7.0.2	The system shall send this request on the reviewer email asking him to add a recommendation for him.
UR7.1	The reviewer shall be able to add his review on the system.
SR7.1.1	The system shall send an email with forwarding link valid until 1 week when the email is sent to the reviewer.
SR7.1.2	The system shall forward the reviewer to a page with information of his name and email filled by the user and can be changed.
SR7.1.3	The system shall allow the reviewer to add his phone number, company and his recommendation letter.
UR7.2	The user shall be able to review the recommendation on email and CV.
SR7.2.1	The system shall send a notification for the user when the reviewer submits the recommendation.
SR7.2.2	The system shall save the information on the user CV.
SR7.2.3	The system shall allow the user to browse the recommendations received from reviewers on his CV.
SR7.2.4	The system shall allow the user to remove any recommendation he doesn't want to keep on his Resume through Resume Builder.

TABLE XI: Resume Builder: Functional Requirements

UR8.0	The user should be able to manage his resume through using resume builder to add and update the user information.
SR8.0.1	The system shall allow the user to view and update his personal information (first name, last name, birth date, email, gender, country, city, mobile, phone optional, photo, facebook url, linkedin url, check if allowing employers to see the personal information).
SR8.0.2	The system shall allow the user to view and update education information (start year, end year, degree type like Diploma, High school, Bachelor, Master, PhD, field of study, minor if exists and university name) where multiple educations can be added, deleted and updated.
SR8.0.3	The system shall allow the user to view and update work experience information (company name, job title, location, from period, to period, description) where multiple experiences can be added, deleted and updated.
SR8.0.4	The system shall allow the user to view and update language information (language name, proficiency scale from 1 to 5) where multiple languages can be added, deleted and updated.
SR8.0.5	The system shall allow the user to view and update skills through skill tag name up to 50 skills where multiple skills can be added, updated and deleted.
SR8.0.6	The system shall allow the user to view and update career objectives which is a long text explaining career future objectives where it can be edited and deleted.
SR8.0.6	The system shall allow the user to view and update certificates information (certificate or course name, date) where multiple certificates can be added, updated and deleted.
SR8.0.7	The system shall allow the user to view and update nationality information (nationality name) where multiple nationalities can be added, updated and deleted.
SR8.0.8	The system shall allow the user to add, delete and view attachments to the resume where multiple attachments can be added, updated and deleted.
UR8.1	The user should be able to preview his resume and download it as a pdf file.
SR8.1.1	The system shall allow the user to preview his resume showing all information in the resume builder the user built (personal information, education, work experience, languages, skills, career objective, certificates and attachments).
SR8.1.2	The system shall allow the user to build the resume as a pdf file that can be downloaded on the device used.

TABLE XII: Employer's Requests: Functional Requirements

UR9.0	The user shall be able to disclose his personal information to employers and remove the permission to view his personal information.
SR9.0.1	The system shall allow the user to disclose his personal information and remove the disclosed permission from employers.
UR9.1	The employer shall be able to request from the user to disclose the personal information.
SR9.1.1	The system shall allow the employer to ask a user to disclose the personal information.
SR9.1.2	The system shall send the employer a notification email when the user discloses his personal information.

TABLE XIII: Employer: Functional Requirements

UR10.0	The employer shall be able to view job applications.
SR10.0.1	The system shall allow the employer all job applications per job post.
SR10.0.2	The system shall allow the employer to view all applications of a specific job post.
SR10.0.3	The system shall order the job applications by Apply date.
UR10.1	The employer shall be able to filter job applications of a specific job post.
SR10.1.1	The system shall allow the employer to filter job applications by title, university, country, degree, job title.
SR10.1.2	The system shall order the job applications by Apply date.

TABLE XIV: Search: Functional Requirements

UR11.0	The users shall be able to search jobs.
SR11.0.1	The system shall allow users to search jobs.
SR11.0.2	The system shall list jobs posts that are relevant to search inquiry.
SR11.0.3	The system shall order job posts based on relevance.
UR11.1	The users shall be able to search articles.
SR11.1.1	The system shall allow users to search articles.
SR11.1.2	The system shall list articles that are relevant to search inquiry.
SR11.1.3	The system shall order articles based on relevance.
UR11.2	The users shall be able to search companies.
SR11.2.1	The system shall allow users to search companies.
SR11.2.2	The system shall list companies that are relevant to search inquiry.
SR11.2.3	The system shall order companies based on relevance.

TABLE XV: Performance: Nonfunctional Requirements

NFR1.0	The system infrastructure should be able to handle scalability dynamically during peak time.
NFR2.0	The system should handle 1,500,000 visitors per year as regular load.
NFR3.0	The system should cache pages that are being heavily requested.

TABLE XVI: Security: Nonfunctional Requirements

NFR4.0	The system should be secured against SQL Injection/JavaScript Injection/ and XSS injections.
NFR5.0	The system should protect public pages with SSL certificate.
NFR6.0	The system should be protected against DDos attaches.